Neural Networks and Transformers

Jelke Bloem

Text Mining
Amsterdam University College

24-03-2026

Announcements

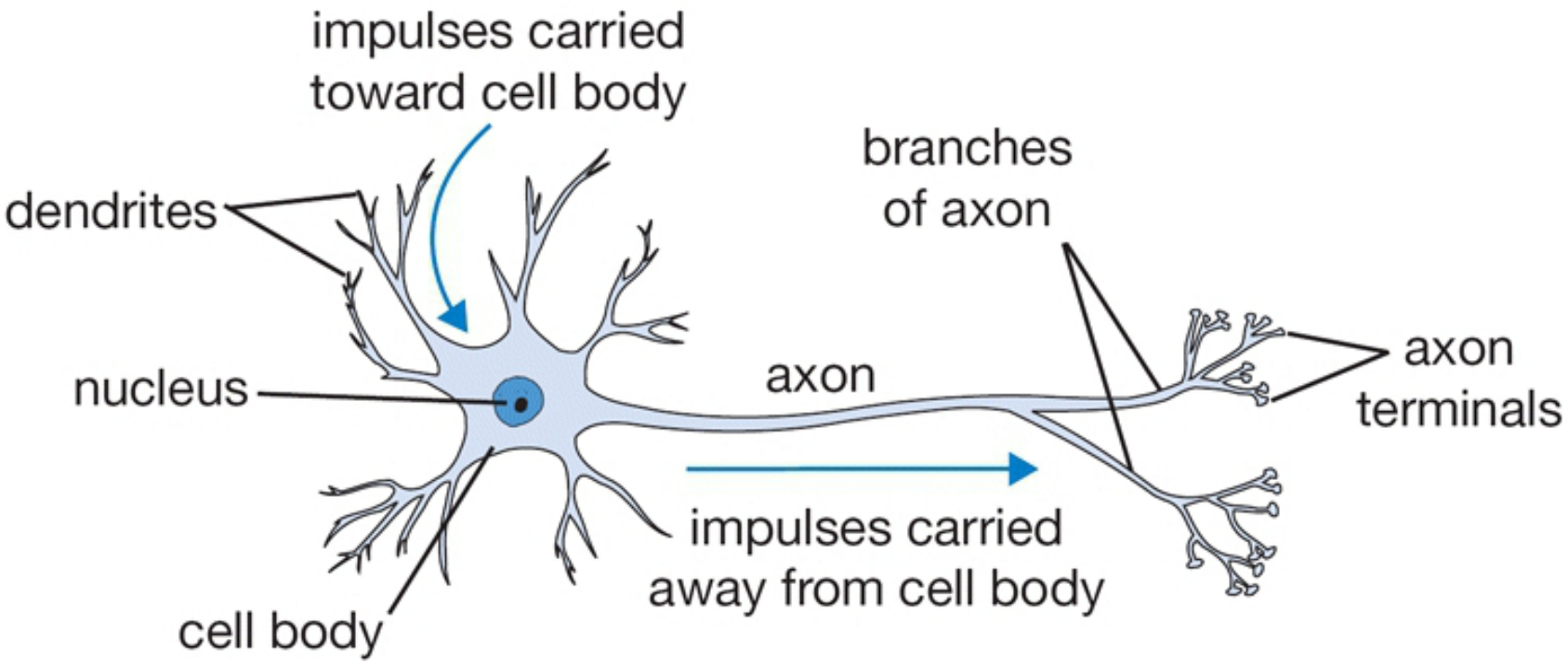
- Assignment 2 due Thursday
- Transformers reading assignment on Friday
- Assignment 3 coming up

Overview

1. Neural Networks
2. Neural Language Models
3. Recurrent Neural Networks
4. Towards Transformer Models
5. Extras

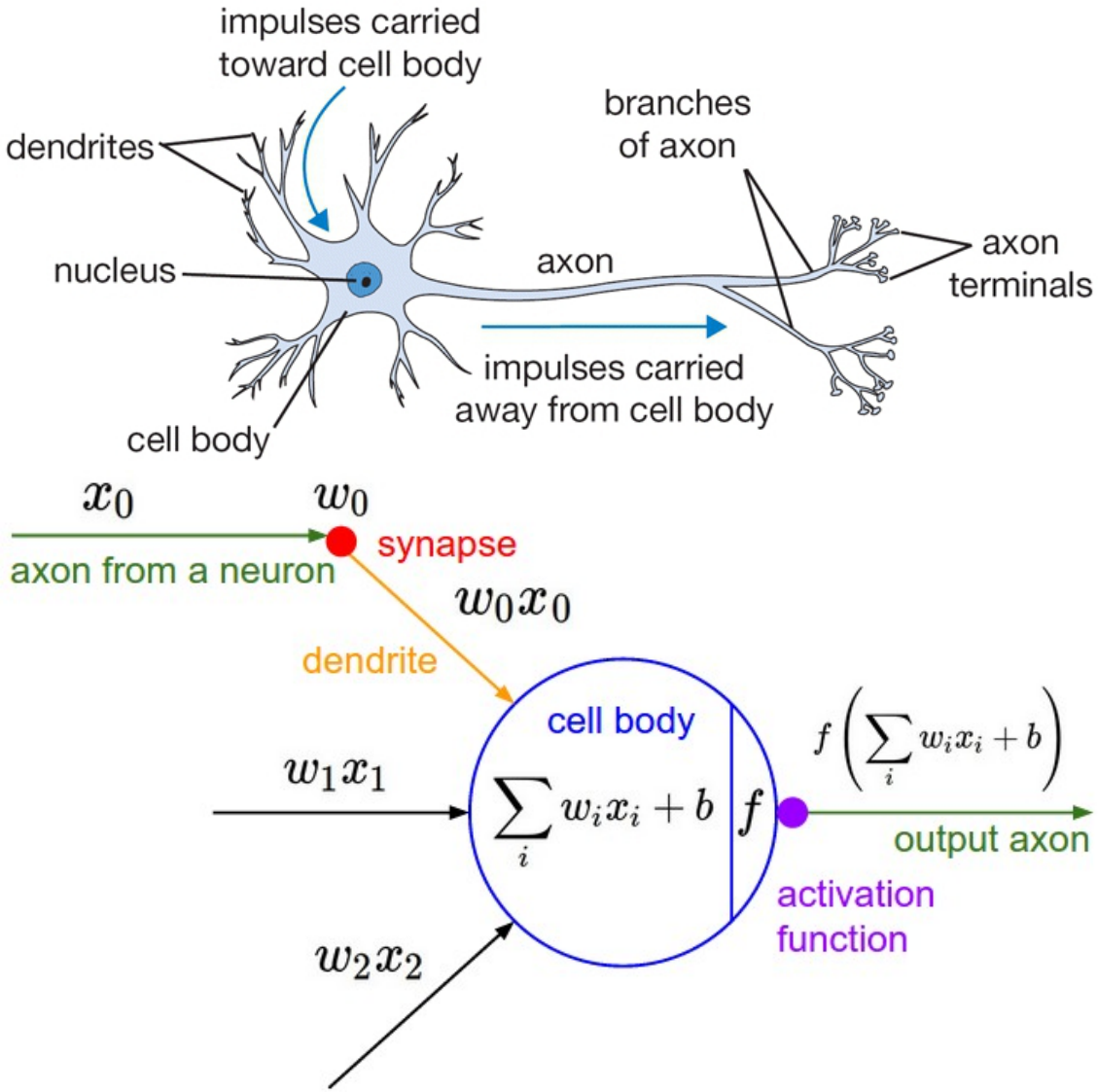
Neural Networks

A single neuron



Credit: Andrej Karpathy via Stanford's CS231N.

A single neuron



Credit: Andrej Karpathy via Stanford's CS231N.

Logistic regression as a neural network

Following the notation in last week's slides, we have:

- $\boldsymbol{x} = \langle x_0, x_1, x_2, \ldots, x_d \rangle$ is our input representation.
- We aggregate the features $\boldsymbol{x}$ into a linear combination using weights $\boldsymbol{w}$. We also include the bias term b into the matrix by adding an appropriate dimension fixed at 1 to $\boldsymbol{x}$, so that we can use matrix notation:

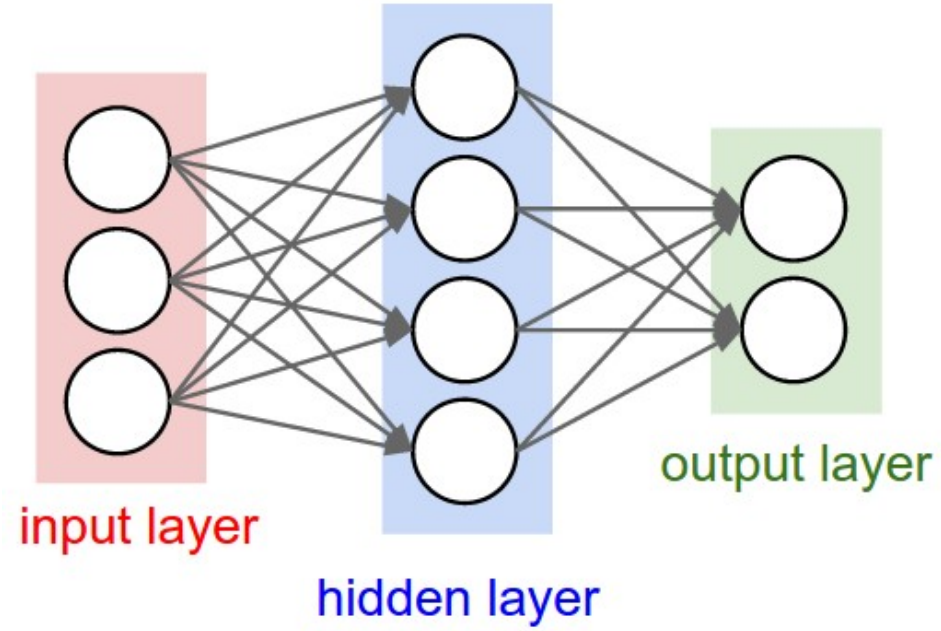
$$z = \sum_{i=0}^{d} w_i x_i = \boldsymbol{w} \cdot \boldsymbol{x}$$

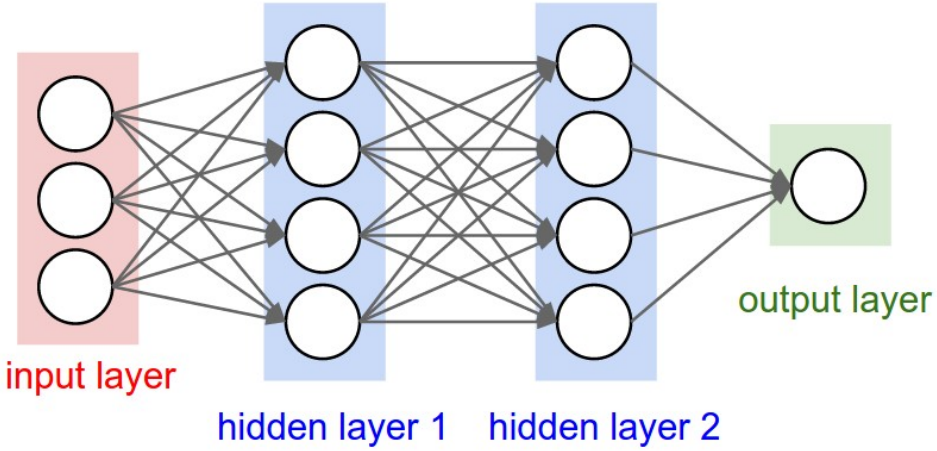
- We pass z through the sigmoid function to map it to range [0,1]:

$$f = \sigma(z) = \frac{1}{1 + e^{-z}}$$

- **Linear models are a single neuron**. *Question: what is the activation function for linear regression?*

From single layer to multi-layer

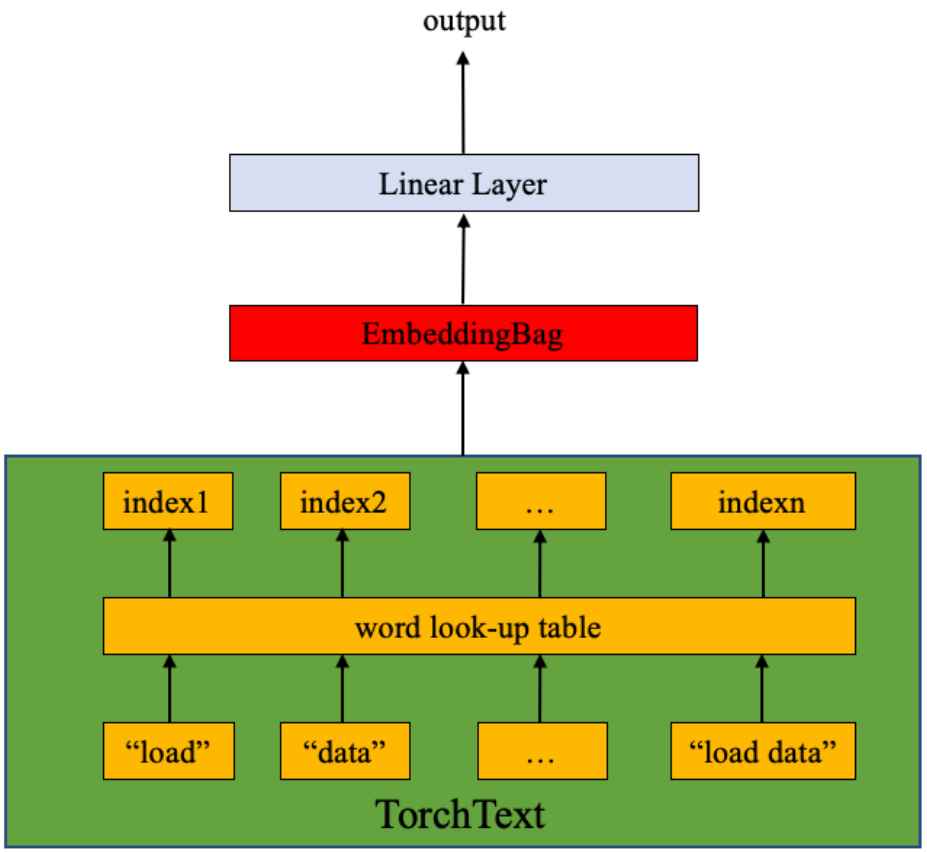




Credit: Andrej Karpathy via Stanford's CS231N.

Using embeddings as features

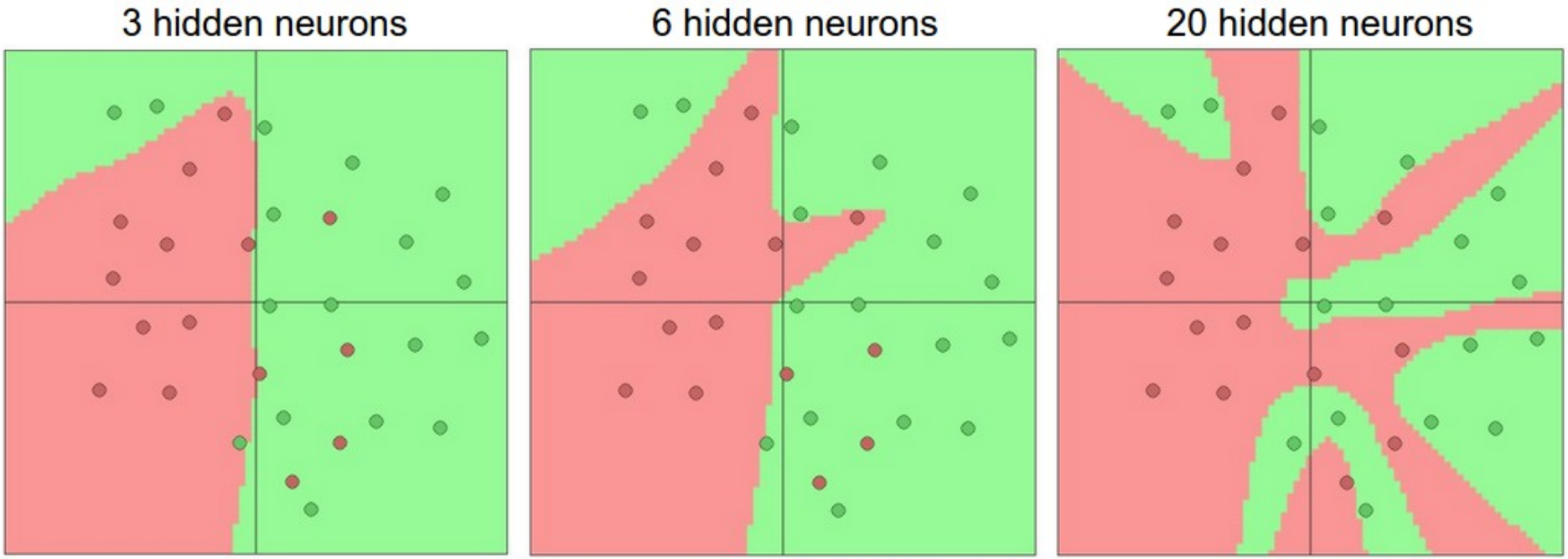
Neural networks are **modular**: we can piece them together into advanced architectures. For example, we can use embeddings to represent our input (either training them or using pre-trained ones). *More on this in the lab.*



Credit: TorchText.

Why do we need non-linearities?

Multiple layers and **non-linear functions** (such as the sigmoid) allow us to fit complex decision boundaries.



Credit: Andrej Karpathy via Stanford's CS231N.

How do we train neural networks?

- Key idea: use a smart way to apply SGD, called **backpropagation**.
- Backpropagation combines using the chain rule to calculate local derivatives (called gradients) with the re-use of pre-computed operations to speed the computation up.
- *More on this in the external materials for the course.*

Neural networks practicalities

Training neural networks entails a lot more than stacking up layers. Several topics require practical and theoretical knowledge beyond this course:

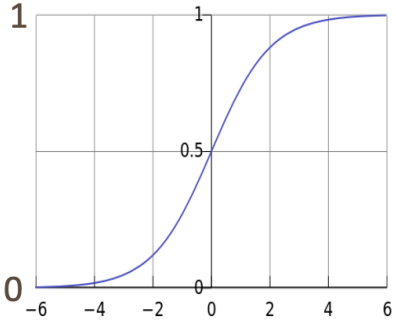
- Weight initialization
- Regularization (e.g., via dropout)
- Which non-linearities to use
- Which loss functions to use
- How to monitor and adjust the learning process (e.g., optimizers and learning rates) to avoid dying neurons and overfitting

NN activation functions

Several non-linear activation functions have been proposed. A good default options is ReLU.

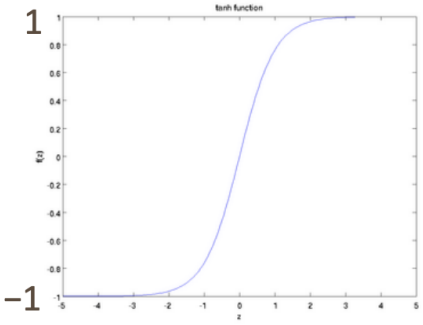
logistic ("sigmoid")

$$f(z) = \frac{1}{1+\exp(-z)}.$$



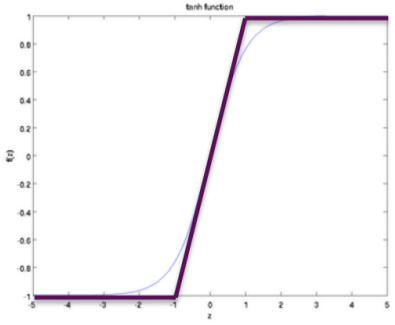
tanh

$$f(z) = \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}},$$



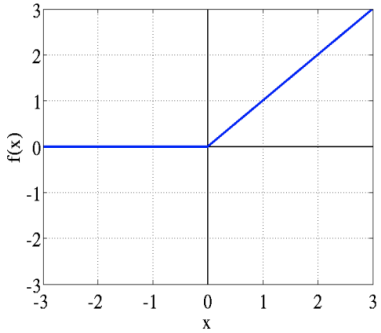
hard tanh

$$\mathrm{HardTanh}(x) = \begin{cases} -1 & \text{if } x < -1 \\ x & \text{if } -1 <= x <= 1 \\ 1 & \text{if } x > 1 \end{cases}$$



ReLU (Rectified Linear Unit)

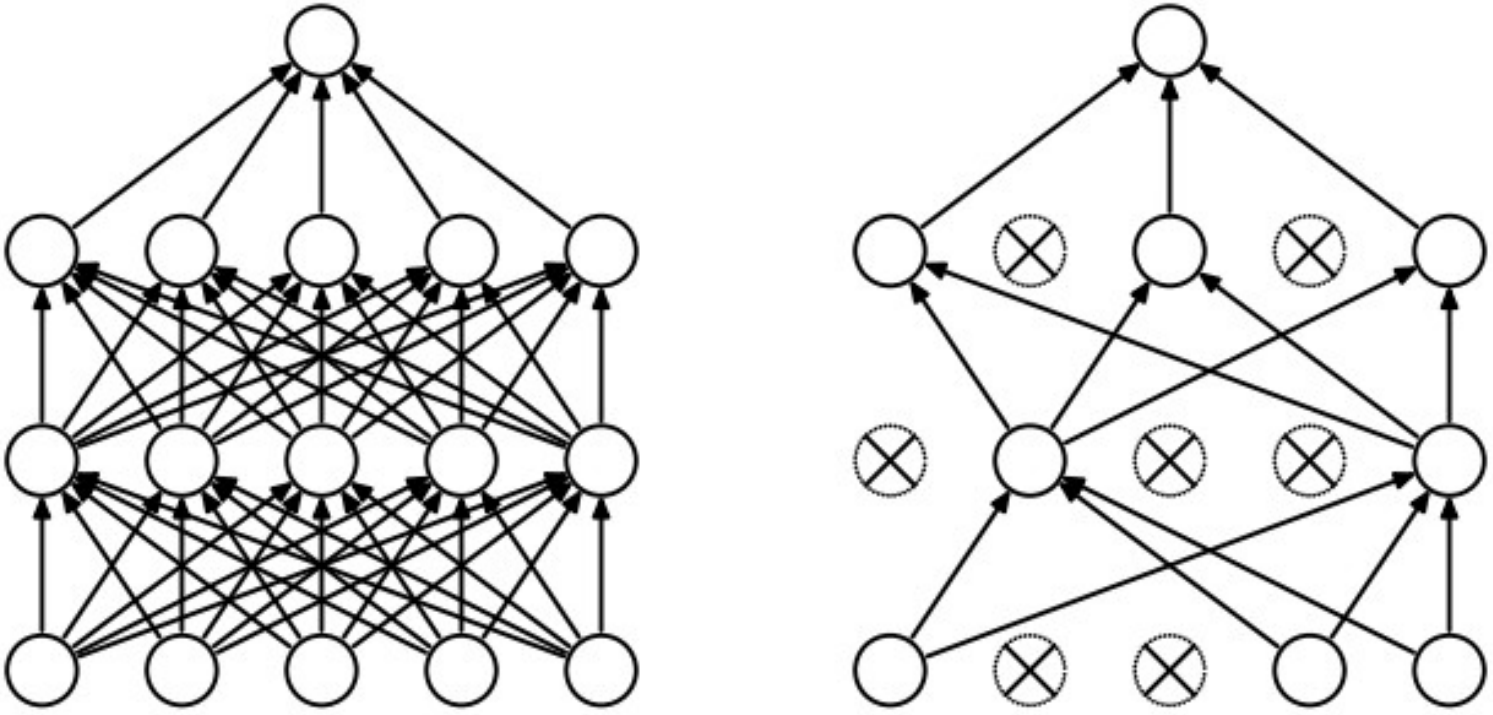
$$\mathrm{rect}(z) = \max(z, 0)$$



Credit: Stanford CS224N.

NN regularization via dropout

Dropout's idea is to mask a random set of neuron connections at training time, in order to compel the network to learn redundant paths and avoid overfitting.

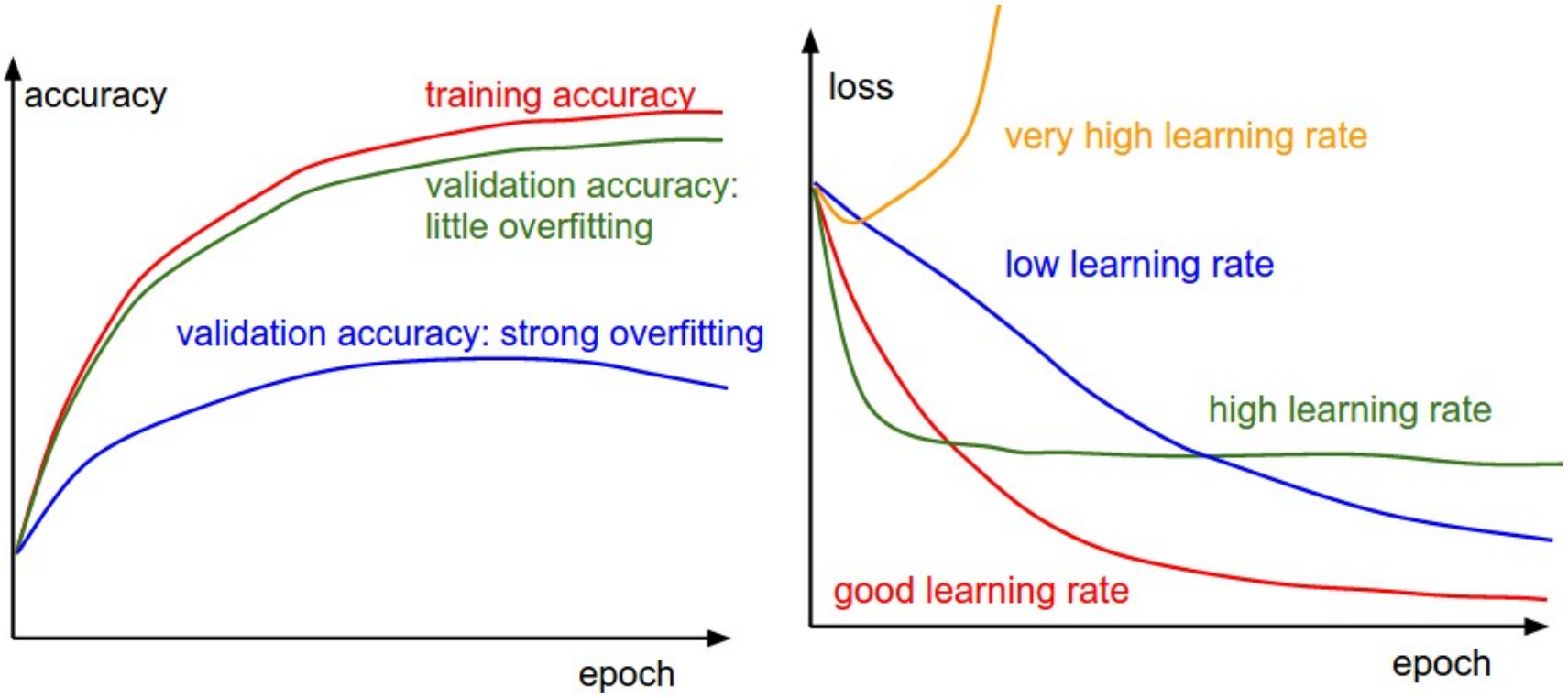


(a) Standard Neural Net (b) After applying dropout.

Credit: Srivastava et al.
https://www.cs.toronto.edu/~hinton/absps/JMLRdropout.pdf.

NN under/overfitting and learning rates

Two illustrations on how to spot correct learning behaviour.



Credit: Andrej Karpathy via Stanford's CS231N.

Word Embeddings reading assignment: Questions

- I am curious as to whether articles that appear often in text like "the" or "a" would influence every token vector of every word and also what the actual vector for these is because they always appear in front of different words.
- Smith stated that "It's interesting to see how the ideas around getting words into computers have come full circle.". I was wondering if (1) this quote referred to the renewed interest in hidden layers for neural networks, and if so, (2) if the 'ideas' he describes are rather afforded by recent technological advances.
  - The circle is context-based meaning
- In Mikolov et al. (2013), the authors explain that RNNs do not perform well on text mining tasks, or at least not as well as BOWs or Skip-grams, mainly due to its complexity. Are there some fields of text-mining where RNN would still be more performant than other models?

Word Embeddings reading assignment: Questions

- Since the CBOW model ignores the word order of the context by treating it as a "bag of words," would the vectors generated by this model show any difference when processing sentences like "dog bites man" and "man bites dog," which have opposite meanings but contain the same words?
- Is it not better for words to have fixed definitions in a system?
- Even though today's model are really developed to capture a word's meaning in its context, I was wondering, how those models would also be able to capture sarcasm in a sentence which would then more be about understanding a sentence as a whole rather than a word in its environment?

Word Embeddings reading assignment: Questions

- Contextual embeddings have been shown to give good context of words but are computationally heavy and require a lot of training data, is there a way to make them more efficient (i.e. less heavy)?
  - DistilBERT (40% smaller), BabyLM
- Why do we not use models with non-linear hidden layers as much anymore?
  - tanh/ReLU, sigmoid in LSTMs are nonlinear. Shift to attention layers
- Since the large language models are based on statistics learned on corpora. Do we think that they could be capable of producing novel ideas?

Neural Language Models

Recap

With language models, we want to compute:

- The probability of a sequence of words: $P(w) = P(w_1, w_2, .., w_n)$.
- The probability of a new word given what came before it: $P(w_n|w_1, w_2, .., w_{n-1})$.
- We have also seen n-gram language models which make use of the Markov assumption. For example, a trigram language model would predict the probability of a sequence of words by conditioning on the two previous words at each step:

$$P(w) = P(w_1)P(w_2|w_1)P(w_3|w_1, w_2) \dots P(w_n|w_{n-2}, w_{n-1})$$

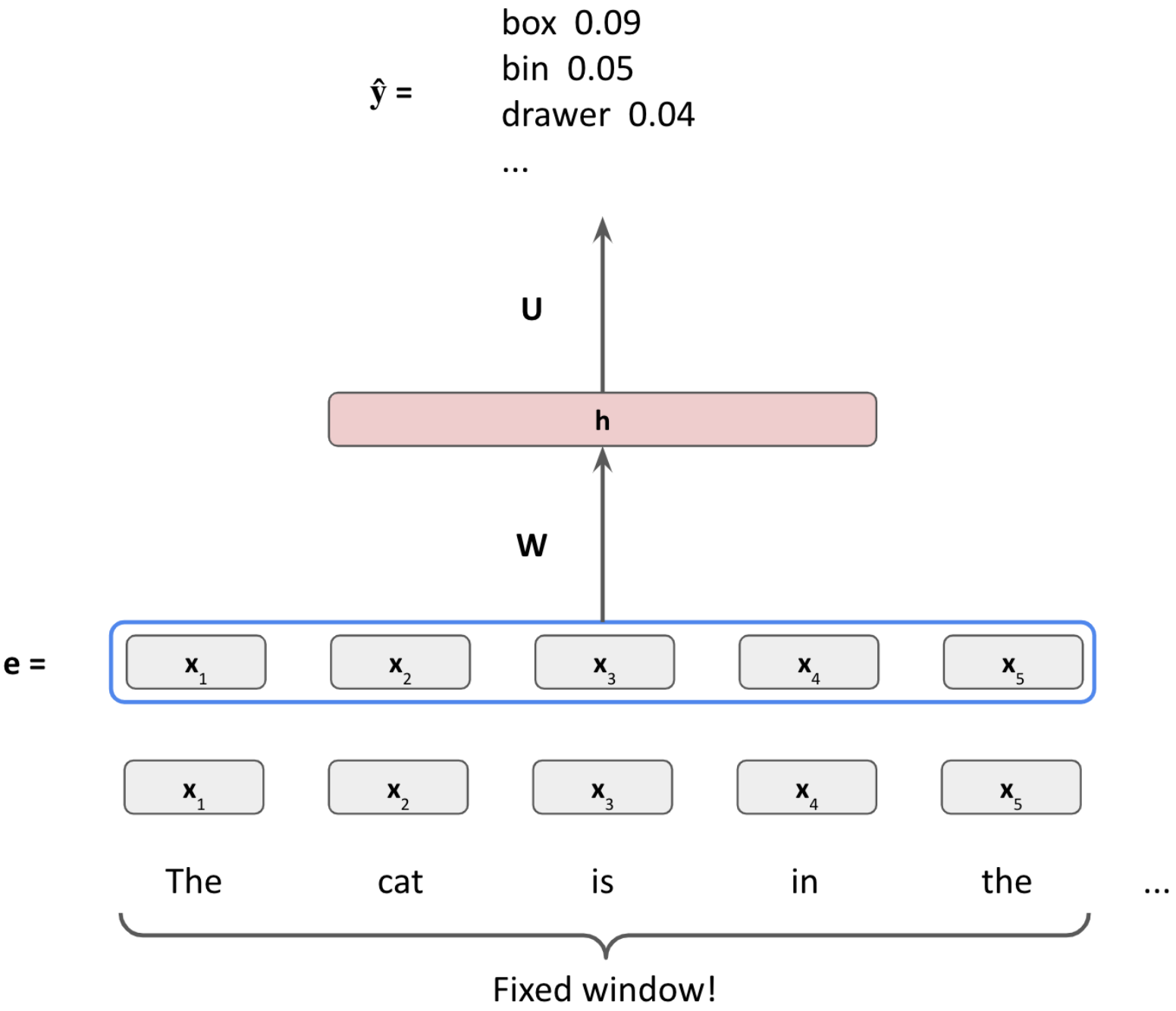
- n-gram language models have some issues, including sparsity and limited use of context.

A first neural language model

We can start by using word embeddings as features for a **fixed-window neural language model**:

- Given a fixed-window sequence of words represented using their embeddings $\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_t$;
- we are interested in estimating the probability of the next word $\hat{p}(\boldsymbol{x}_{t+1} | \boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_t)$.
- We can use the previous words' embeddings as features, concatenate them and feed them to a hidden layer with a non-linearity, and concluding by estimating probabilities with a softmax.

A first neural language model



A first neural language model

Where:

- Each $\boldsymbol{x}$ is a $1 \times d$ embedding vector of dimensionality d.
- $\boldsymbol{e} = [\boldsymbol{x}_1; \boldsymbol{x}_2; \ldots; \boldsymbol{x}_t]$ is the concatenation of the embeddings of the words in the fixed-window preceding w_{t+1}. Therefore $\boldsymbol{e}$ has dimensionality $1 \times dk$, where k is the number of words in the fixed-window.
- $\boldsymbol{h} = f(\boldsymbol{W}\boldsymbol{e}^T)$ is the hidden layer, with f an appropriate activation function and $\boldsymbol{W}$ a weight matrix. $\boldsymbol{W}$ has dimensionality $dk \times h$ and $\boldsymbol{h}$ has dimensionality $1 \times h$. The dimensionality of the embeddings and the hidden layer are hyperparameters of the model.
- Finally, $\hat{y} = softmax(\boldsymbol{U}\boldsymbol{h}^T)$ is the predicted next word probability as estimated using a softmax function. Here $\boldsymbol{U}$ is another weight matrix of dimensionality $|V| \times h$, so that with the softmax we predict a probability distribution over the vocabulary V.

A first neural language model

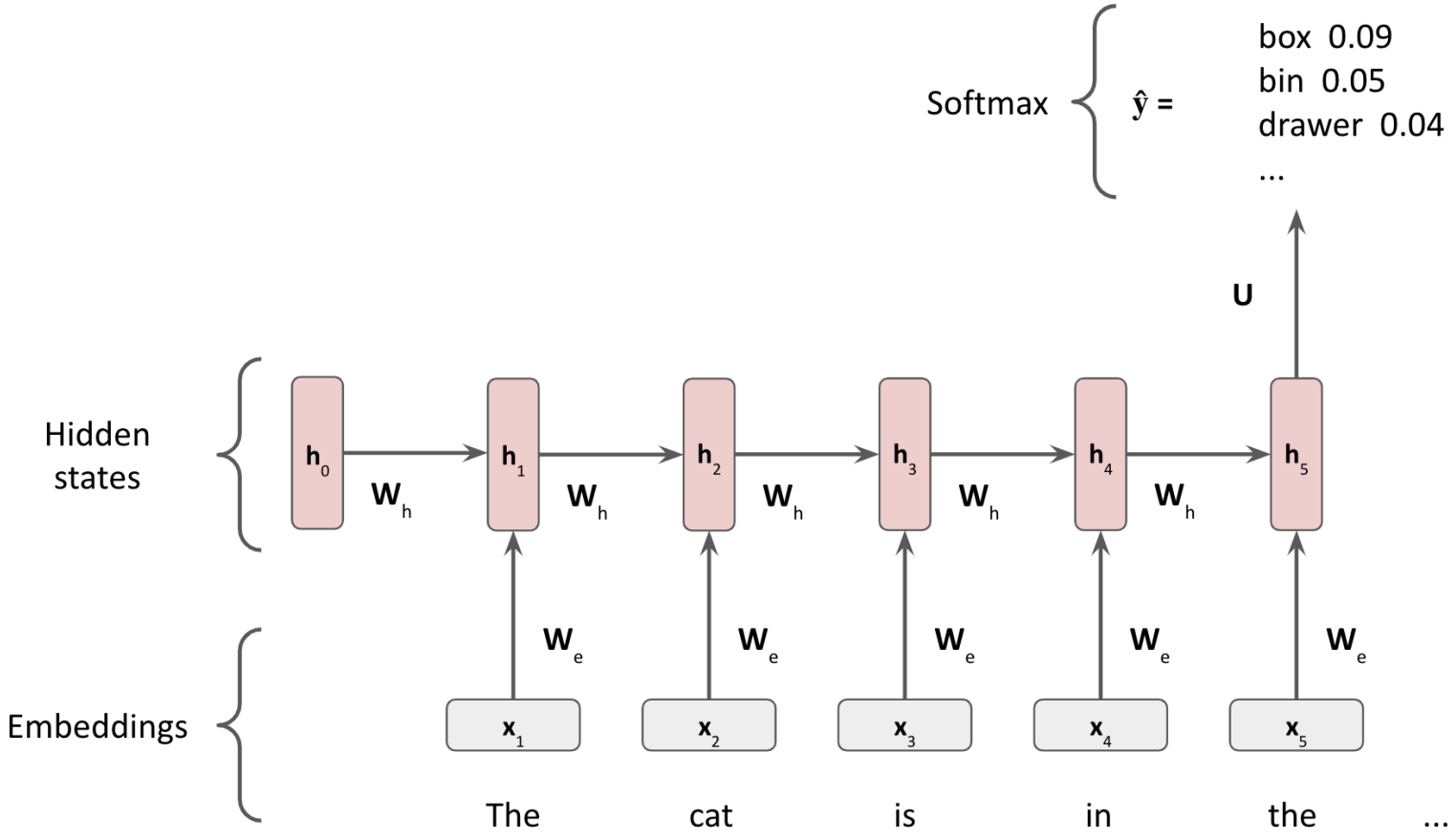
This model is interesting, yet unfortunately:

- it does not solve the use of a broader context with flexible word windows;
- larger windows would mean more parameters so scale is also an issue;
- and it does not make an efficient use of parameters and shared weights.

Can we do better? Yes, with **Recurrent Neural Networks**.

Recurrent Neural Networks

Recurrent architecture



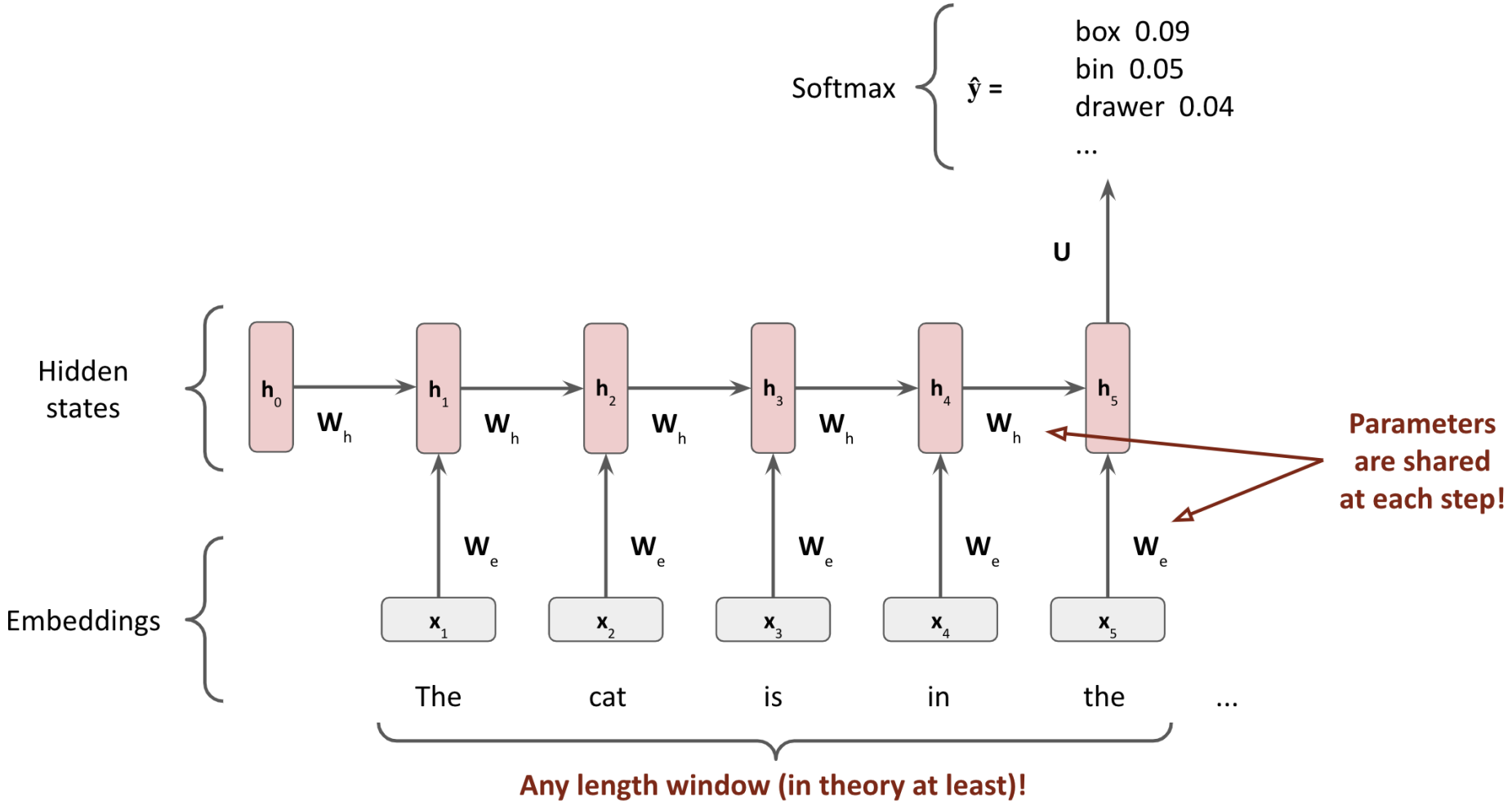
Recurrent architecture

Where:

- Each $\boldsymbol{x}$ is a $1 \times d$ embedding vector of dimensionality d.
- $\boldsymbol{h}_t = f(\boldsymbol{W}_h \boldsymbol{h}_{t-1}^T + \boldsymbol{W}_e \boldsymbol{x}_t^T)$ is the hidden layer, with f an appropriate activation function, $\boldsymbol{W}_e$ and $\boldsymbol{W}_h$ weight matrices. $\boldsymbol{W}_h$ has dimensionality $h \times h$, $\boldsymbol{W}_e$ has dimensionality $h \times d$, and $\boldsymbol{h}$ has dimensionality $1 \times h$. $\boldsymbol{h}_0$ is the initial hidden layer. The dimensionality of the embeddings and the hidden layer are hyperparameters of the model.
- Finally, $\hat{y} = softmax(\boldsymbol{U}\boldsymbol{h}^T)$ is the predicted next word probability as estimated using a softmax function. Here $\boldsymbol{U}$ is another weight matrix of dimensionality $|V| \times h$, so that with the softmax we predict a probability distribution over the vocabulary V.

(Notebook 8.1 Part II: Model)

Recurrent architecture



Why RNNs?

RNNs are a big step forward re. our previous concerns:

- Can process inputs on any length and use previous context of any length (in theory);
- model size does not depend on window size ($\boldsymbol{W}$ matrices remain of the same dimension);
- weights are shared across time steps.

Nevertheless, RNNs can be slow and won't really remember information from many steps back.

Training RNNs

How can we train RNNs?

- Predict the next word at each step, and calculate the loss accordingly.
- The loss at step t is the usual **cross-entropy** (now on multiple classes), calculated for the word to be predicted at $t+1$:

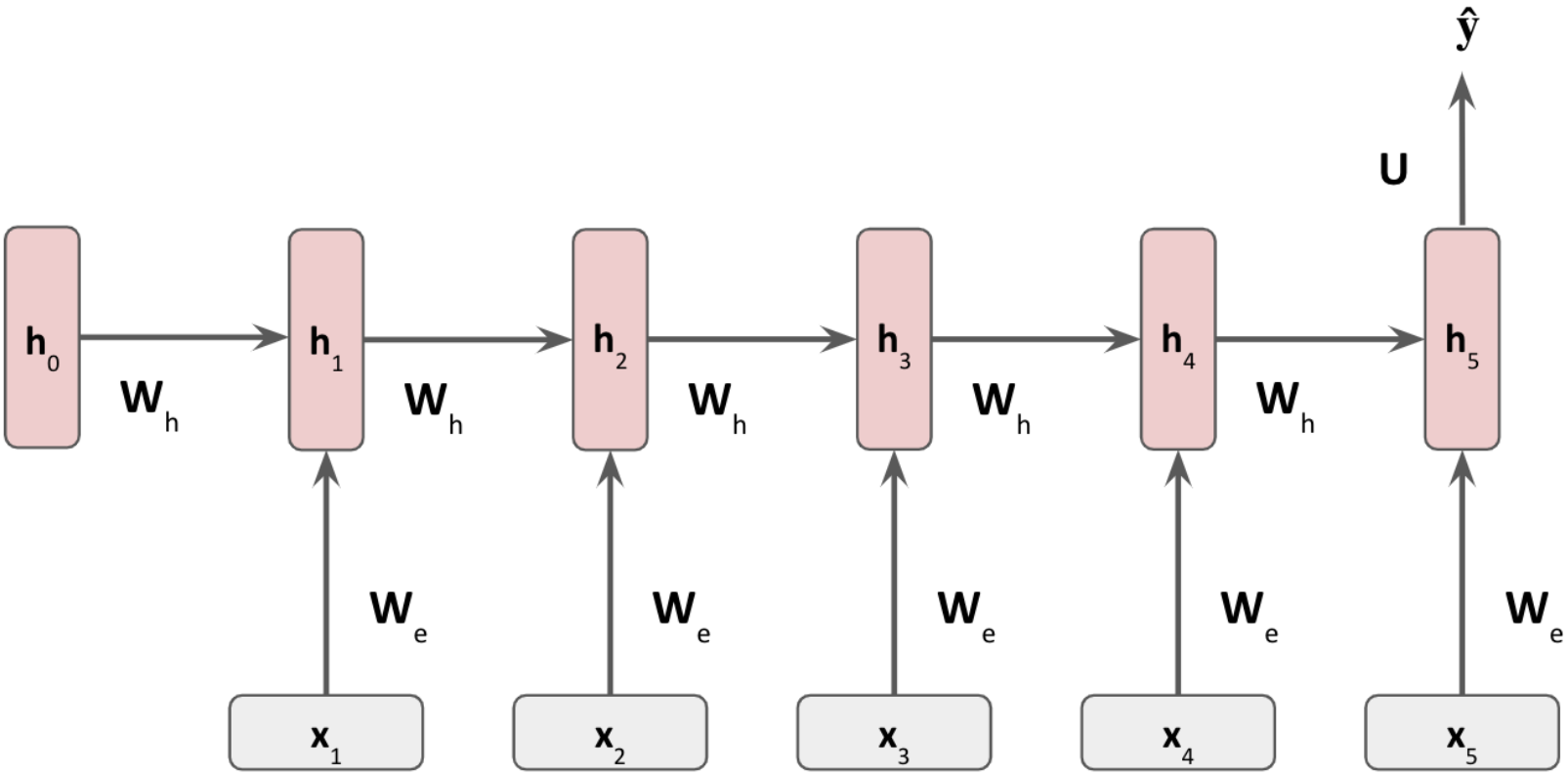
$$\mathcal{L}_t(\boldsymbol{W}) = -\sum_{w \in V} y_{w_{t+1}} log(\hat{y}_{w_{t+1}}) = -log(\hat{y}_{w_{t+1}})$$

- The overall loss is the average of the sum of the losses, calculated at each step:

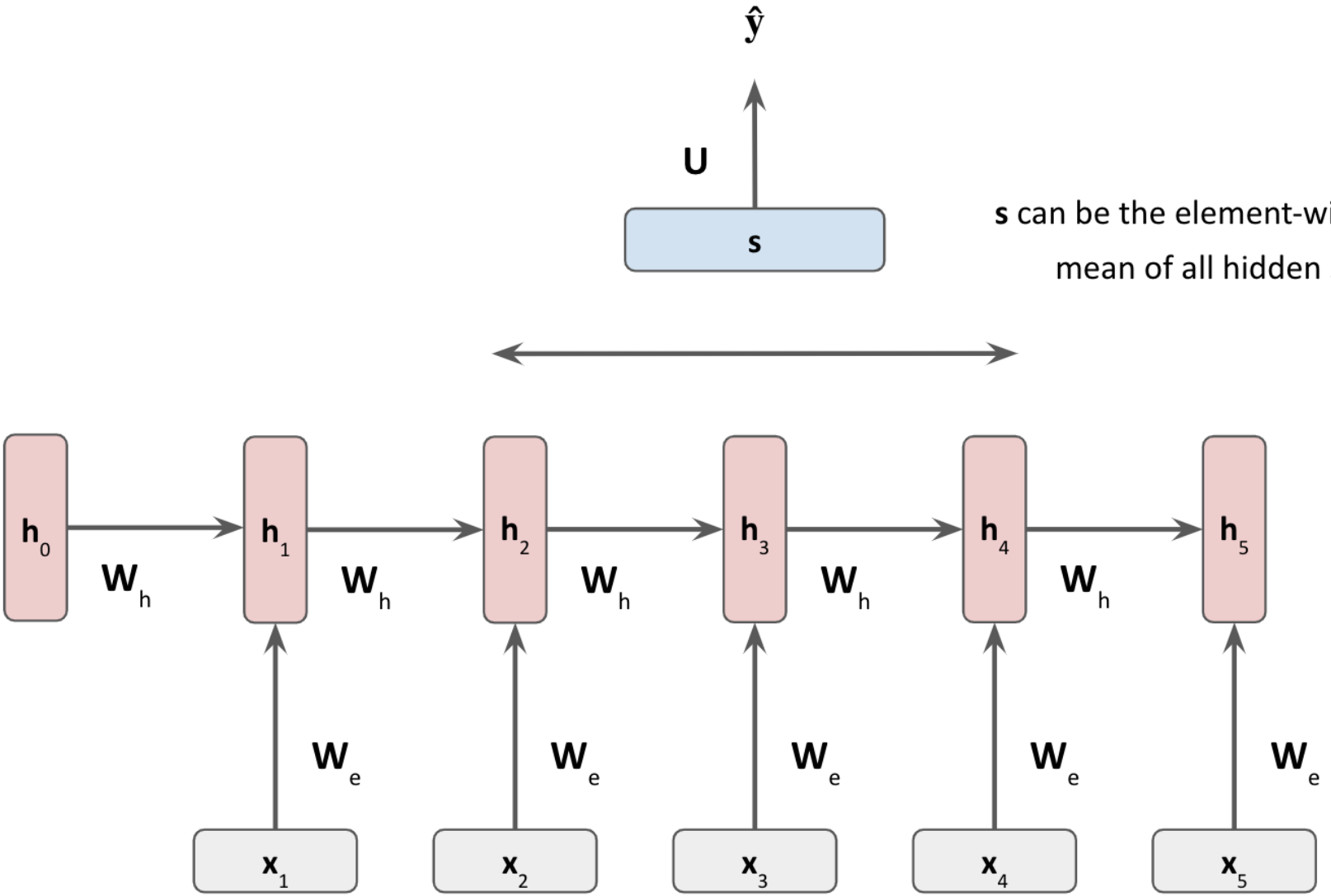
$$\mathcal{L}(\boldsymbol{W}) = \frac{1}{T}\sum_{t=1}^{T} \mathcal{L}_t(\boldsymbol{W}) = \frac{1}{T}\sum_{t=1}^{T} -log(\hat{y}_{w_{t+1}})$$

- Optimization can be done via SGD (backpropagation). Since the parameters $\boldsymbol{W}$ are used repeatedly, this is sometimes called **backpropagation through time**.

RNN flavors: Many to one

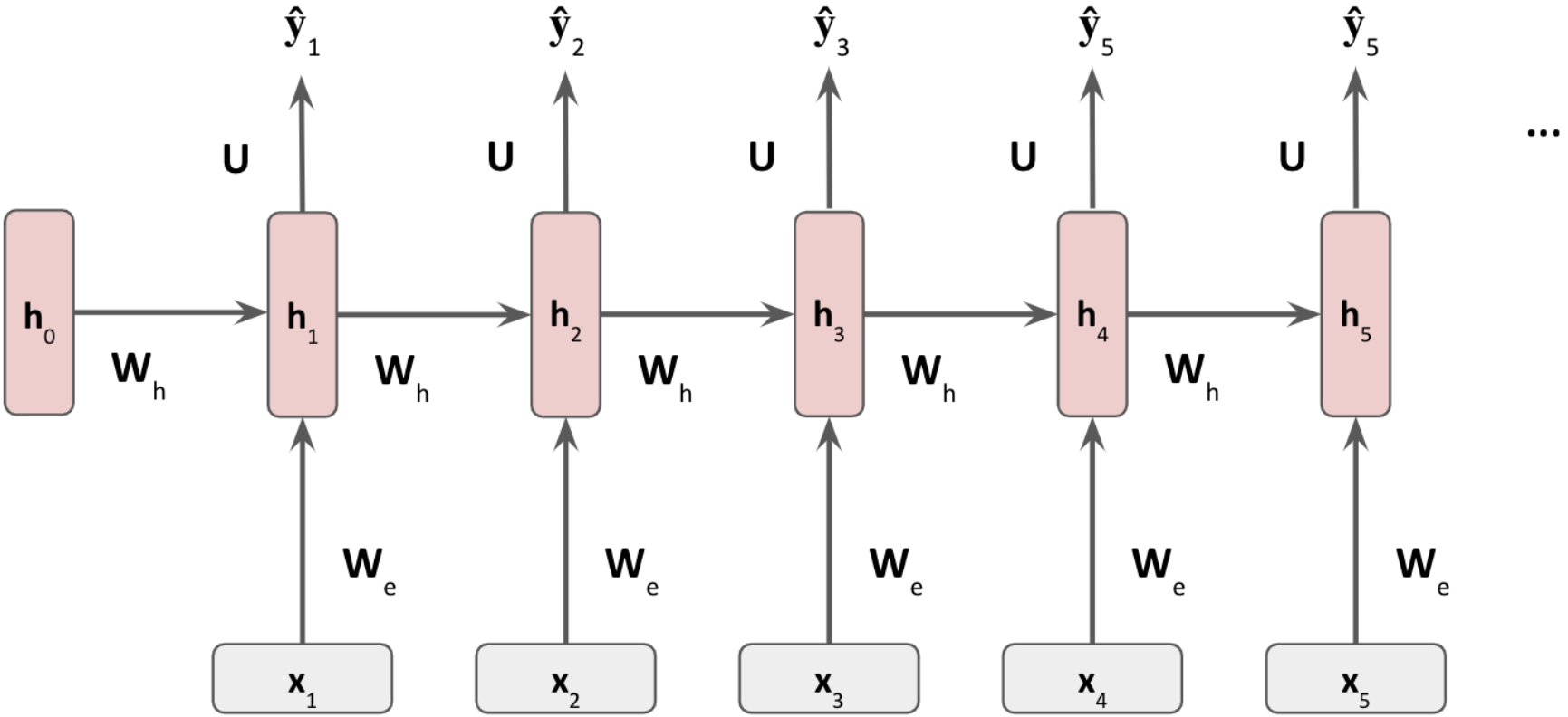


RNN flavors: Many to one



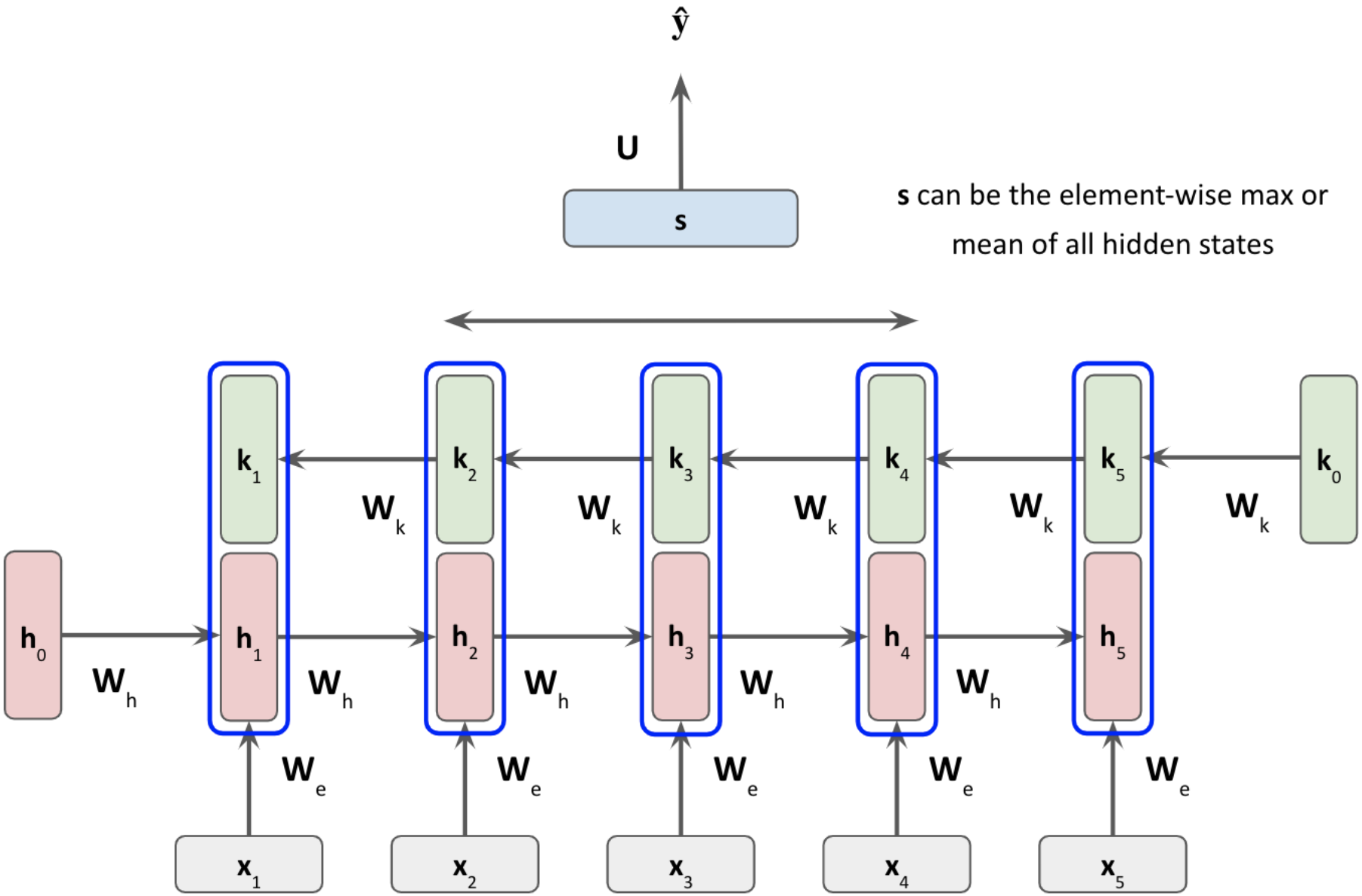
s can be the element-wise max or mean of all hidden states

RNN flavors: Many to many

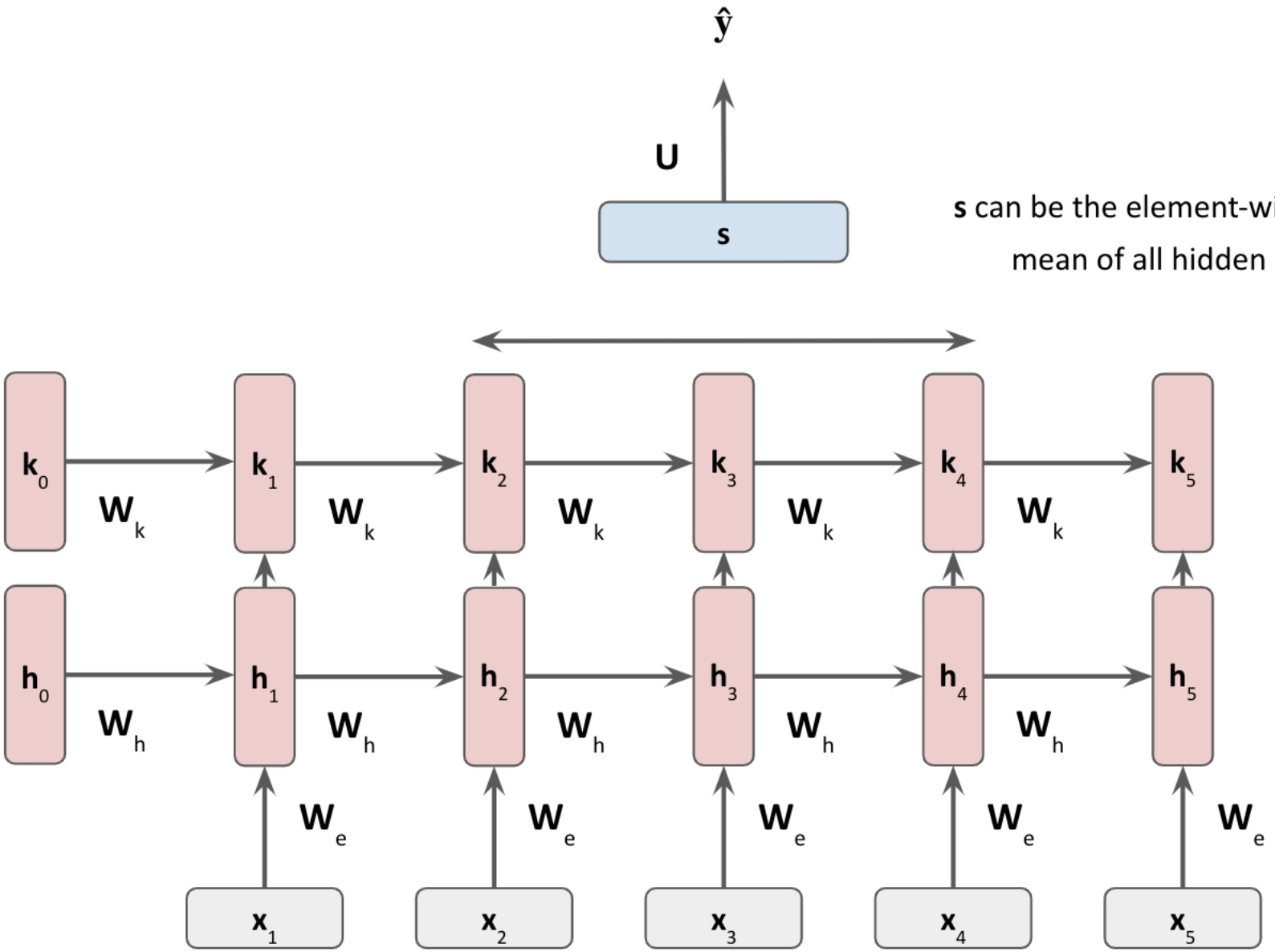


Towards Transformer Models

Bi-RNNs


ŷ
U
s
s can be the element-wise max or mean of all hidden states
k1
k2
k3
k4
k5
k0
Wk
h0
h1
h2
h3
h4
h5
Wh
We
x1
x2
x3
x4
x5

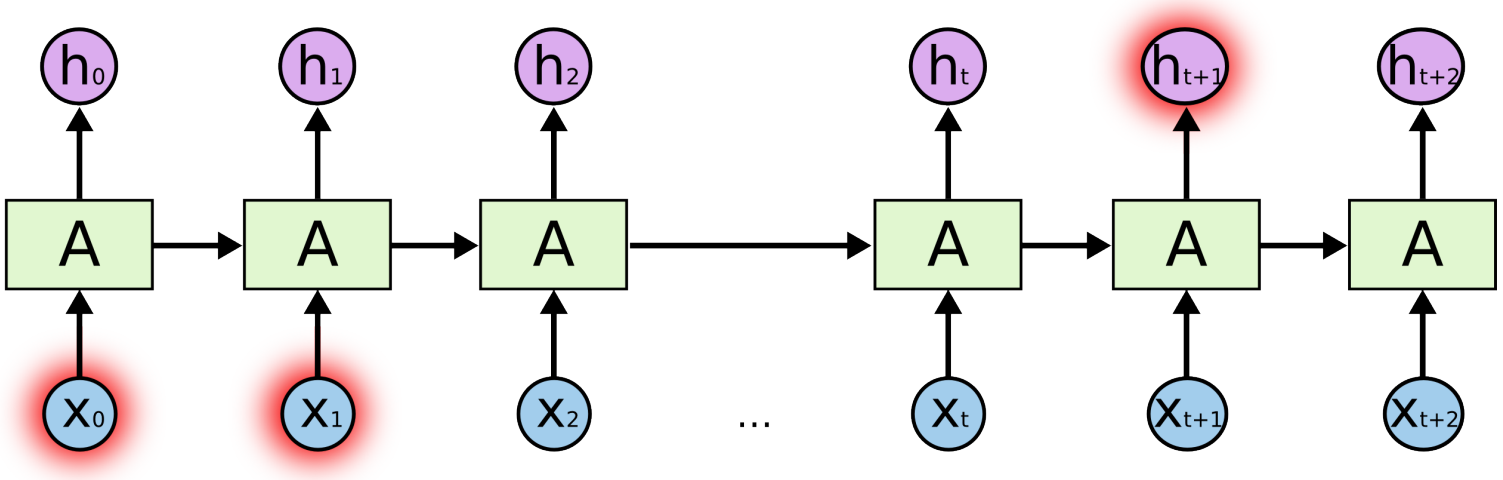
Multilayer RNNs



s can be the element-wise max or mean of all hidden states

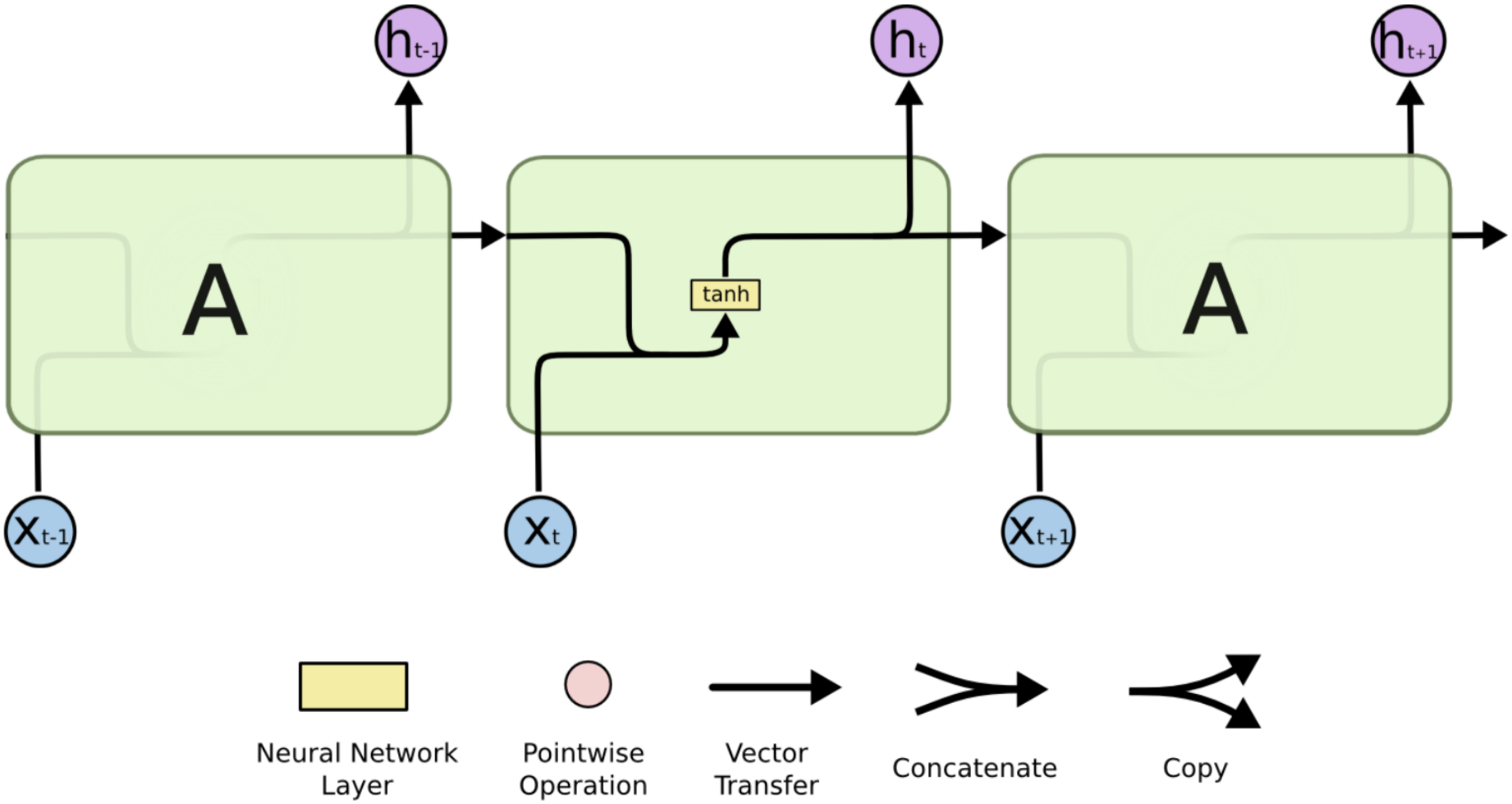
Vanishing Gradients

- RNNs have a crucial issue: **vanishing and exploding gradients**.
- Both occur when we backpropagate through time with multiplying several times by $\boldsymbol{W}$. If $\boldsymbol{W}$'s parameters are small, gradients can vanish to zero. If they are large, they can explode.
- This is an issue as it does not allow to model far away context (vanishing) or to properly converge (exploding).
- Solutions:
  1. Exploding: **gradient clipping**.
  2. Vanishing: **Long Short-Term Memory** networks.



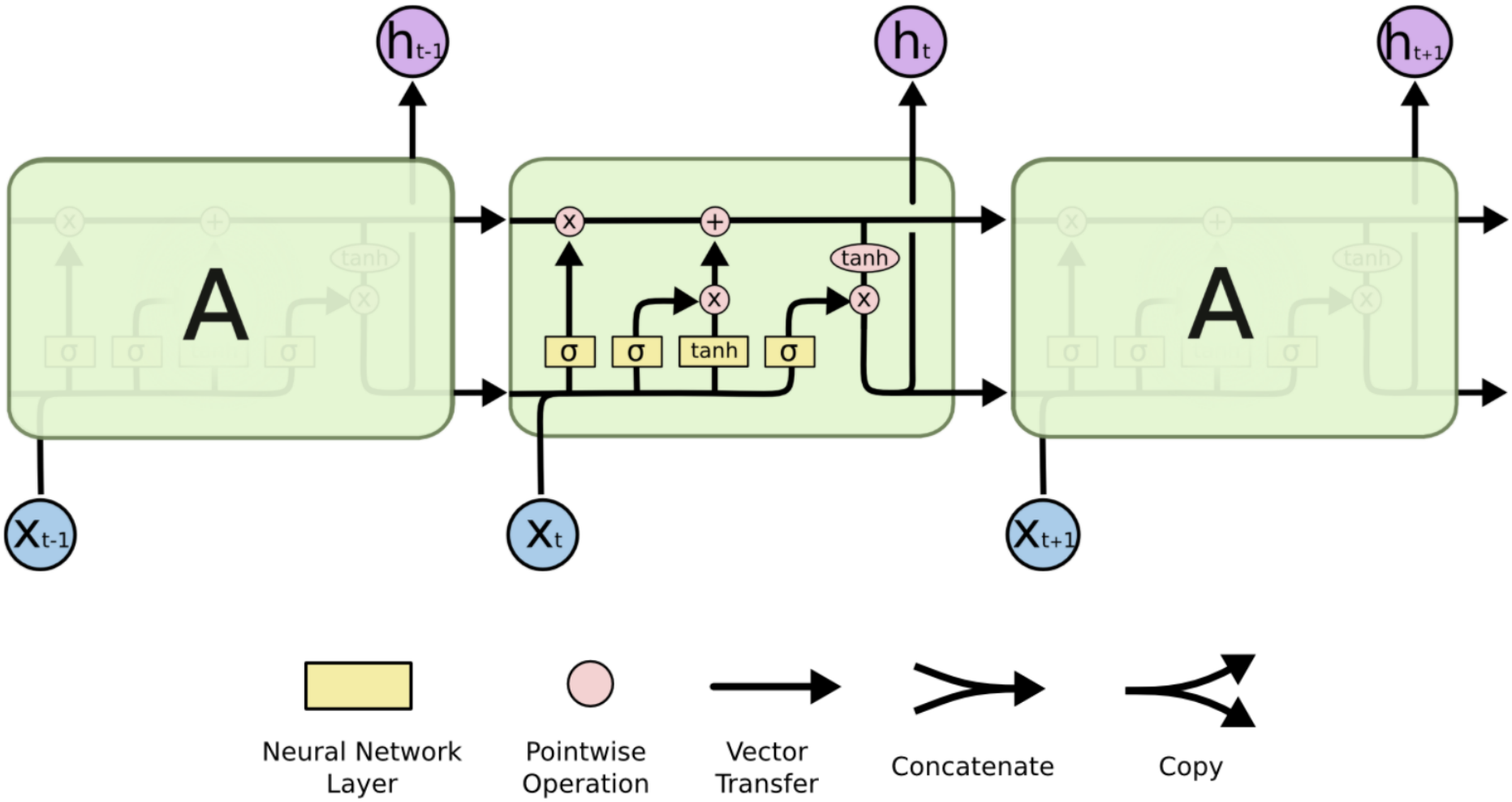
Credit: `https://colah.github.io/posts/2015-08-Understanding-LSTMs`

A different view on RNNs



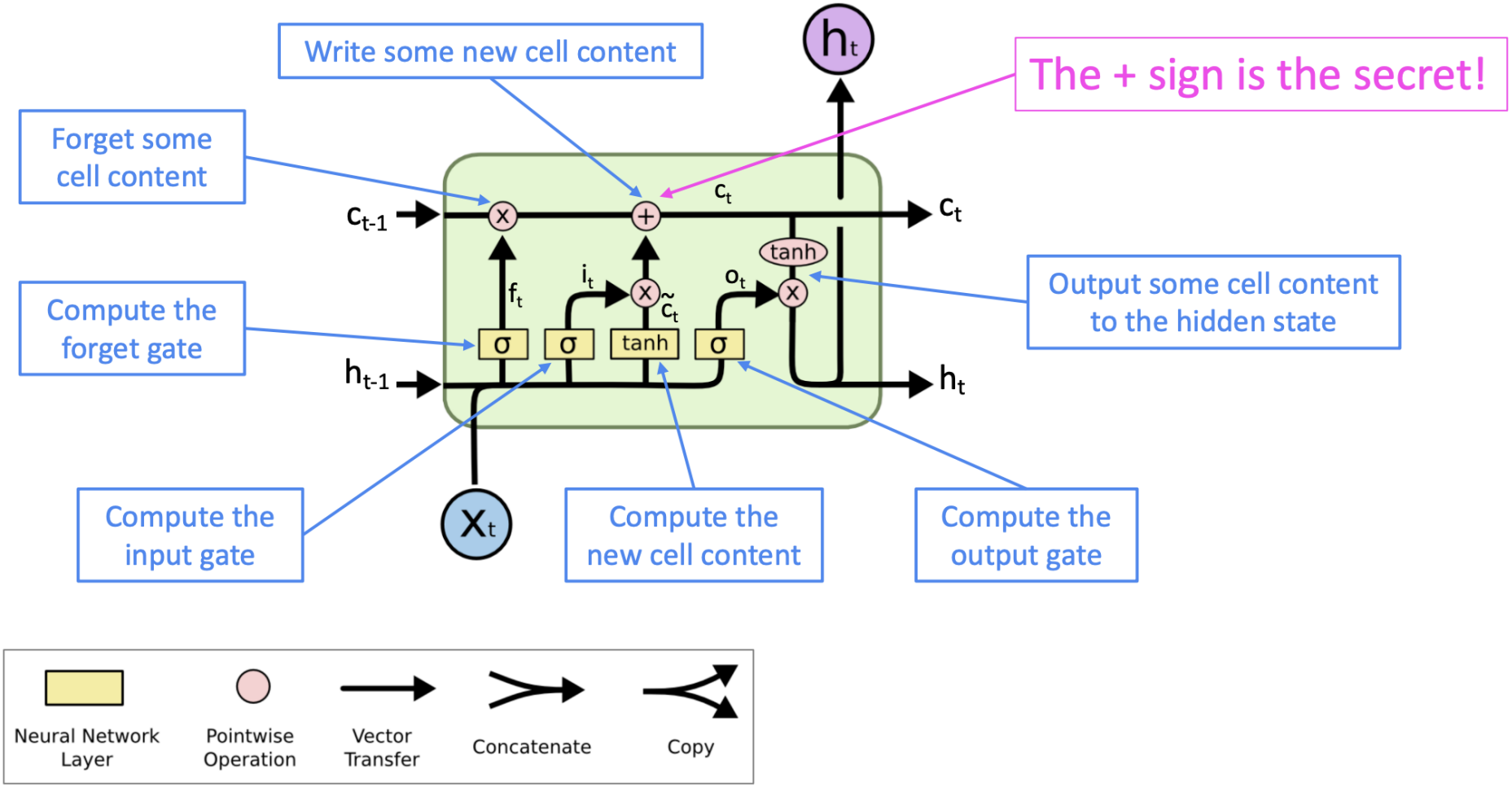
Credit: `https://colah.github.io/posts/2015-08-Understanding-LSTMs`

LSTMs



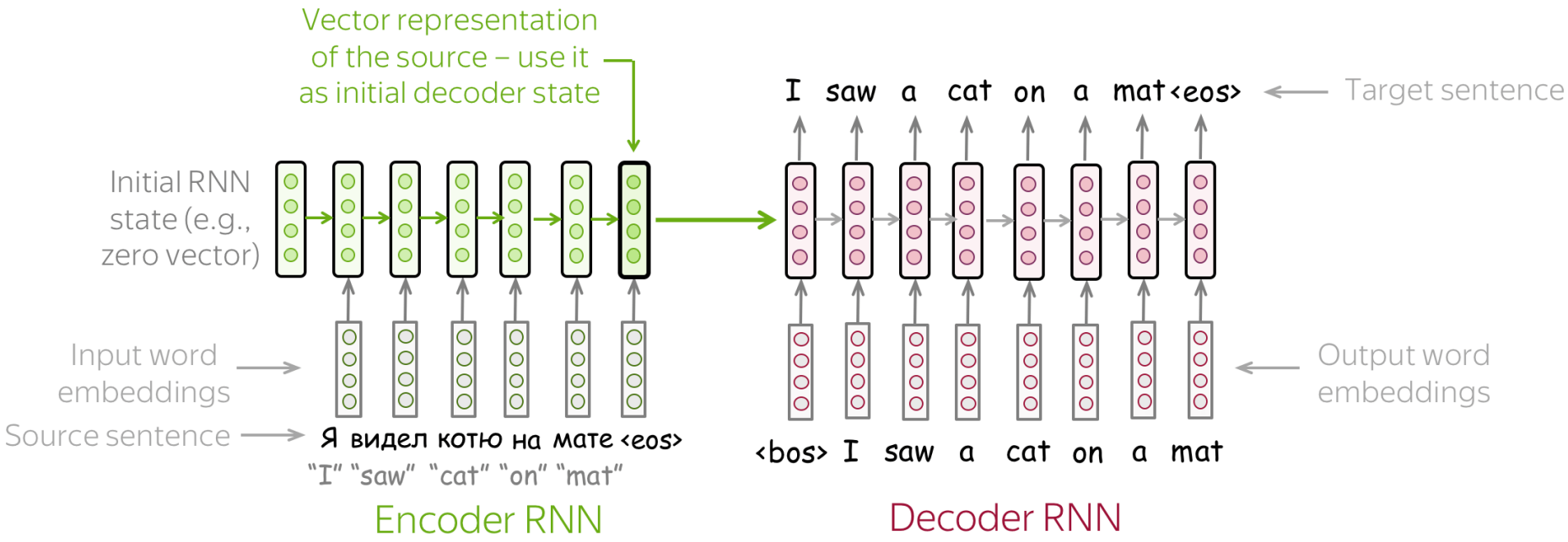
Credit: `https://colah.github.io/posts/2015-08-Understanding-LSTMs`

LSTMs



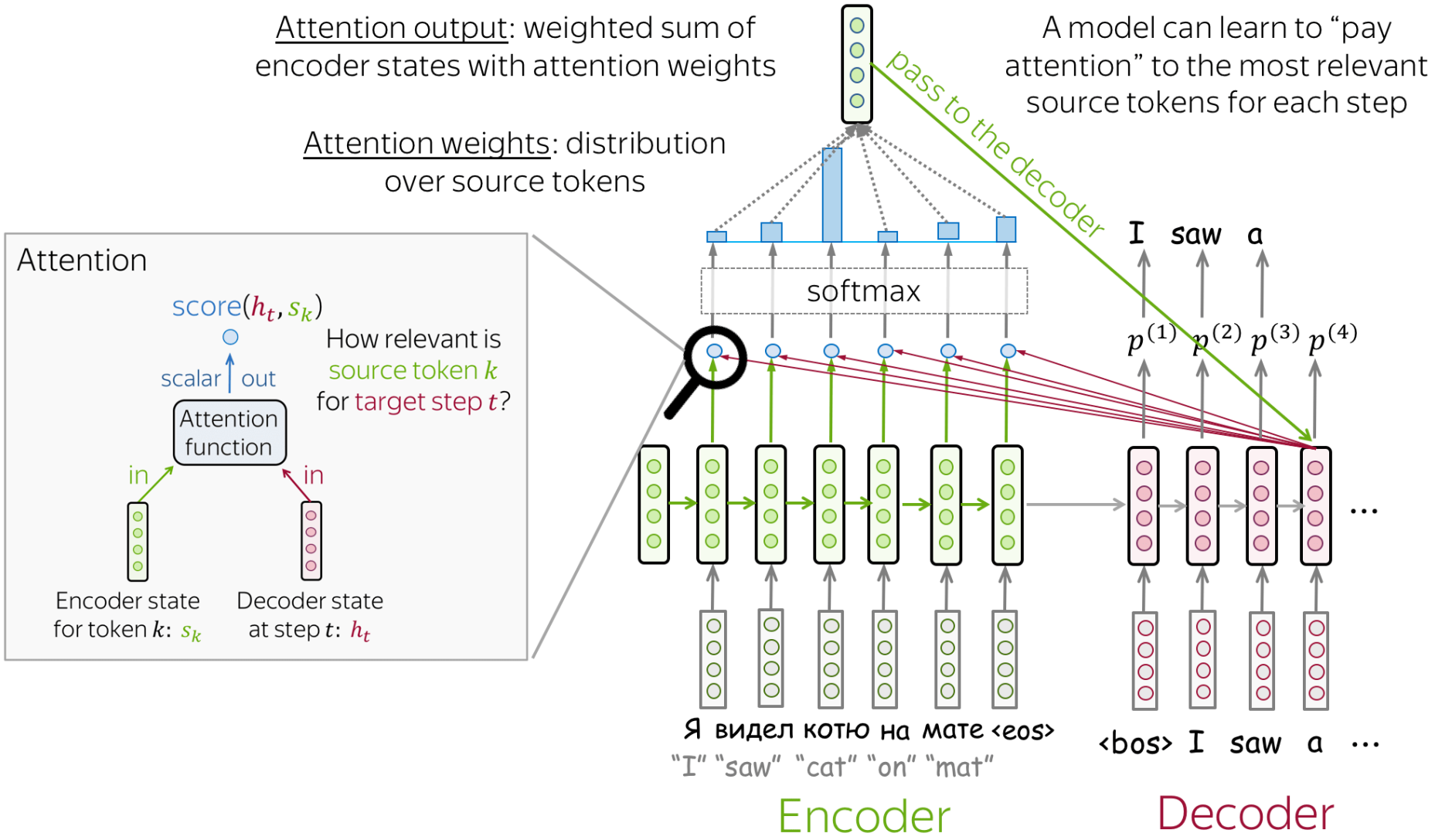
Credit: Stanford CS224N

Seq2Seq



Credit: Lena Voita `https://lena-voita.github.io/nlp_course.html`

Attention



Credit: Lena Voita `https://lena-voita.github.io/nlp_course.html`

What's next

- The most recent advances in neural networks for NLP come from the shift from recurrent architectures to using **attention-based architectures**.
- For your reading assignments, you have explored **transformers** (which combine attention with other ideas from neural networks literature) and will explore **BERT** (which is a neural language model making use of transformers).
- While there is much more to it, in this way you will have a window into contemporary models for NLP.

The next part of the course turns to other topics instead: Web scraping and APIs, recommender systems, corpus annotation, sentiment analysis and clustering with topic modelling, ethics.

Extras

Example: The perceptron

- The very beginnings (Rosenblatt 1958), still at the core for the neural model of learning.
- Inspired by a neuron with incoming connections from other neurons that represent features
- We have a **dataset** $\{X, Y\}$, so that $\boldsymbol{x} = \langle x_1, x_2, \ldots, x_d \rangle, y \in \{-1, +1\}, \forall (\boldsymbol{x}, y) \in \{X, Y\}$. We thus have a binary classification problem.
- The **perceptron model** is defined by a linear combination of weights $\langle w_1, w_2, \ldots, w_d \rangle$ and features, plus an optional bias term:

$$a = \left[\sum_{d=1}^{D} w_d x_d \right] + b$$

- *sign* is our **classification function** (positive or negative): does the neuron fire? The bias shifts the decision threshold.

Perceptron loss

- Training a perceptron is online: exemplar by exemplar
- When do we have to adjust the weights?
- We can use the so-called **0/1 loss**.
  $l^{0/1} = \min_{\boldsymbol{w},b} \sum_n \mathbf{1}[y_n(\boldsymbol{w} \cdot \boldsymbol{x}_n + b) < 0]$
- Equivalently, to simplify: $l^{0/1} = \min \sum_n \mathbf{1}[y_n \hat{y} < 0]$
- For each mistaken prediction, penalty of 1
- When the data are linearly separable, the minimum can be zero. *Why?*

Perceptron optimization

- How does the perceptron learn?

$$a = \left[\sum_{d=1}^{D} w_d x_d \right] + b$$

- **Optimization**: it is an online (1 data point at the time) and error-driven algorithm (we want to make no errors). Given a datapoint in the train dataset, *if the perceptron's prediction is correct, do nothing, else*:

$$w_d^t \leftarrow w_d^{t-1} + y x_d$$
$$b^t \leftarrow b^{t-1} + y$$

(y is the label, positive or negative)

Exercise

Exercise: Training a perceptron

Your dataset is the following: $\{(2,1;-1),(1,2;+1),(3,1;-1),(3,2;-1),(1,3;+1),(2,3;+1)\}$. Assume we use a perceptron without bias term. Also assume we start with random weights: $w_1^{(0)} = 1/2, w_2^{(0)} = -1/2$.

- The first iteration goes as follows:
  1. $a_1 = w_1^{(0)} x_{11} + w_2^{(0)} x_{12} = 1 - 1/2 = 1/2$. $sign(1/2) = +$, thus we have an error and we need to update weights.
  2. Weight update at iteration 1:

$$w_1^{(1)} \leftarrow w_1^{(0)} + y_1 x_{11} = 1/2 - 2 = -3/2$$
$$w_2^{(1)} \leftarrow w_2^{(0)} + y_1 x_{12} = -1/2 - 1 = -3/2$$

  3. Proceed to do the same for w_2 and the following data points. Does your perceptron converge to a boundary after one pass on the data? If so, can you draw the boundary?

Exercise

Exercise: Training a perceptron

1. $a_2 = w_1^{(1)} x_{21} + w_2^{(1)} x_{22} = -3/2 - 3 = -4.5.$ $sign(-4.5) = -$

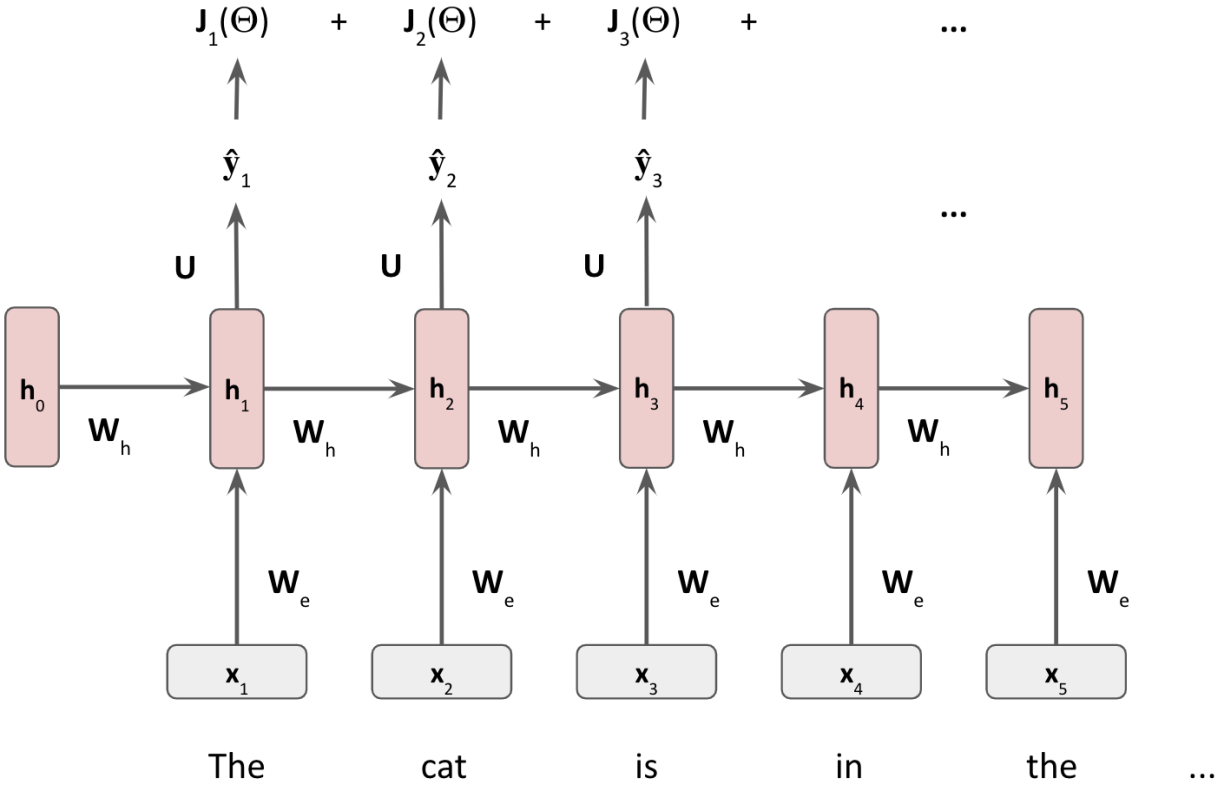
$$w_1^{(2)} \leftarrow w_1^{(1)} + y_1 x_{11} = -3/2 + 1 = -1/2$$
$$w_2^{(2)} \leftarrow w_2^{(1)} + y_1 x_{12} = -3/2 + 2 = 1/2$$

2. $a_3 = w_1^{(1)} x_{31} + w_2^{(1)} x_{32} = -3/2 + 1/2 = -1.$ $sign(-1) = -$
3. $a_4 = w_1^{(1)} x_{41} + w_2^{(1)} x_{42} = -3/2 + 1 = -1/2.$ $sign(-1/2) = -$
4. $a_5 = w_1^{(1)} x_{51} + w_2^{(1)} x_{52} = -1/2 + 3/2 = 1.$ $sign(1) = +$
5. $a_6 = w_1^{(1)} x_{61} + w_2^{(1)} x_{62} = -2/2 + 3/2 = 1/2.$ $sign(1/2) = +$

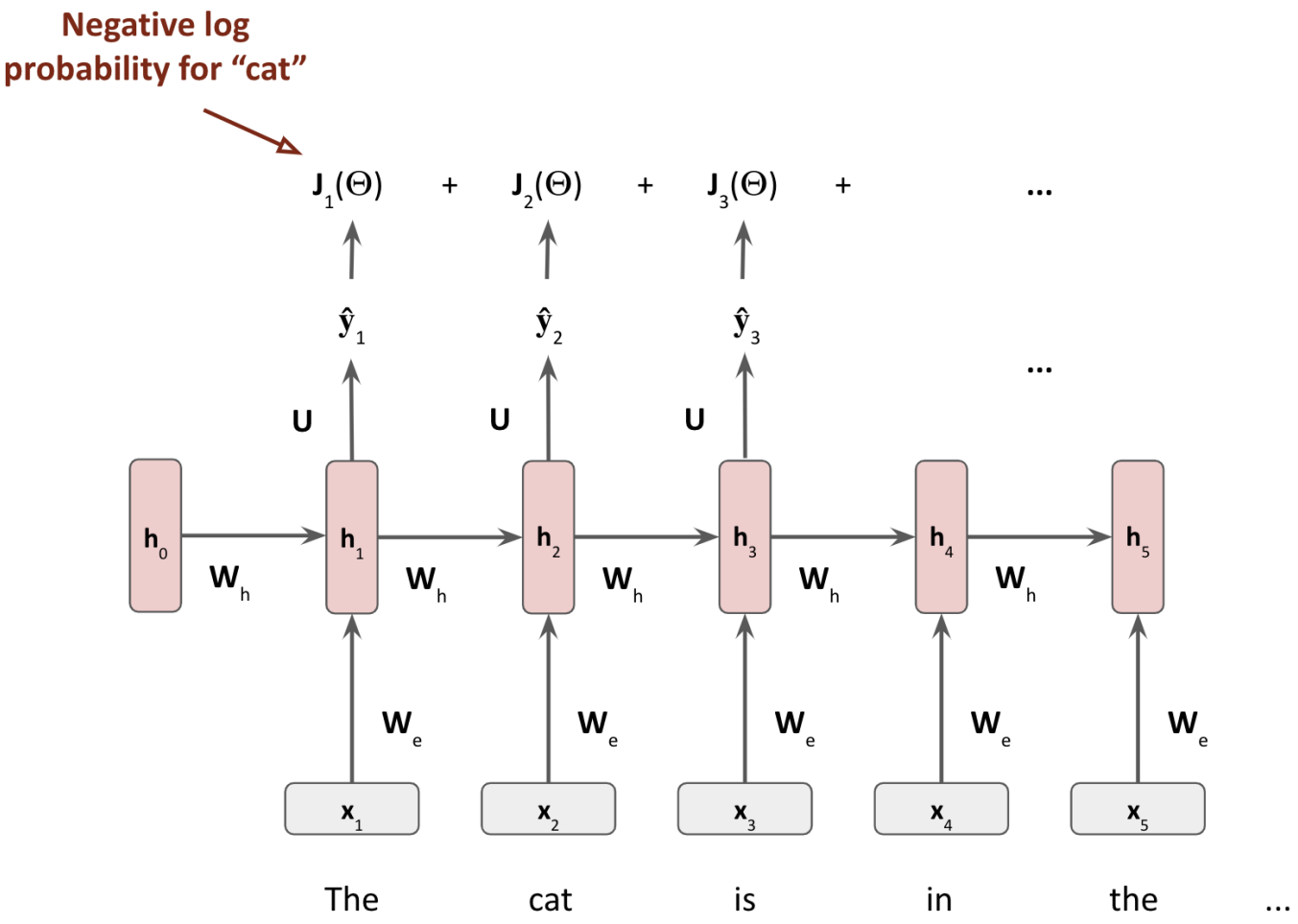
In summary

- Some properties of the perceptron include:
  1. **It always converges if the data points are linearly separable**.
  2. It is unable to distinguish among decision boundaries.
  3. The linear model it embeds computes **a projection of every feature** x_d onto the vector $\boldsymbol{w}$. This means that we basically order the projected features on a line, sum them up and check if they are above or below a threshold!
  4. It is unable to go beyond linearly separable data (infamous XOR problem). Extensions include: 'stacking up' perceptrons (**neural networks**) and doing feature maps (**kernel methods**).
  5. See HD, ch. 4 for more.

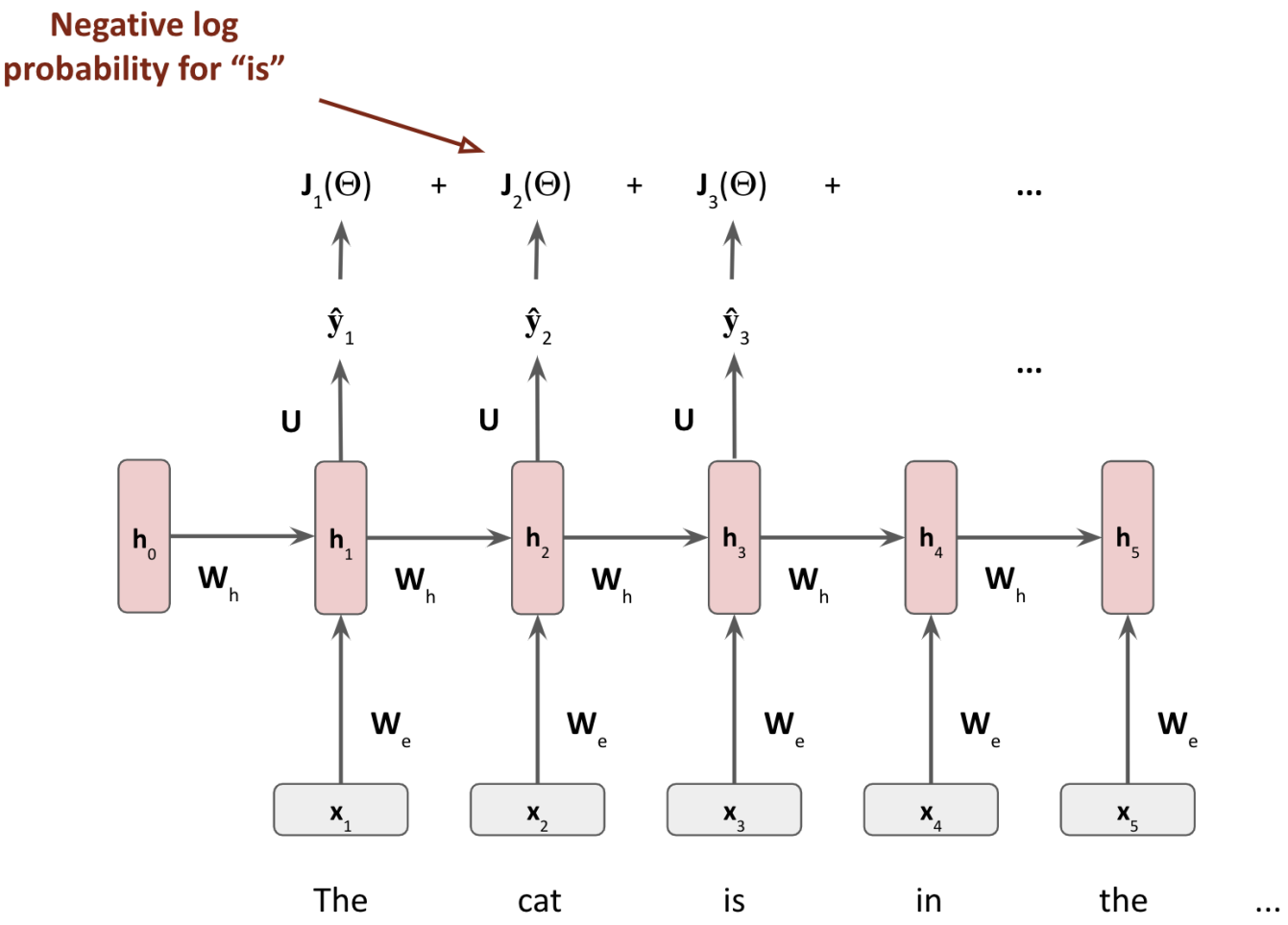
Training RNNs



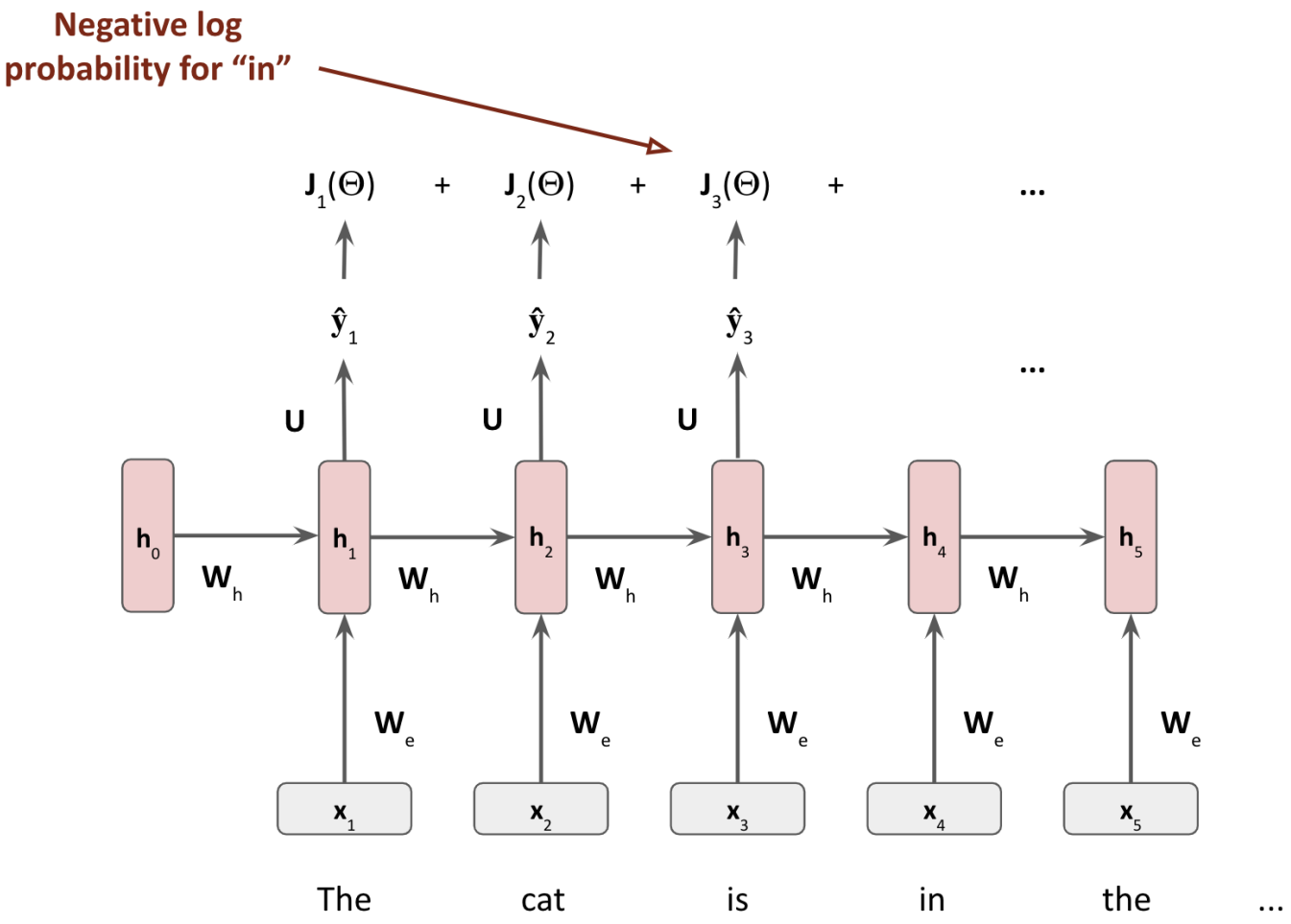
Training RNNs



Training RNNs



Training RNNs



Recall perplexity?

A second look at perplexity:

- We defined it as the inverse probability of the corpus, normalized by the number of words. For a corpus composed of n words:

$$PP = P(w_1, w_2, \ldots, w_n)^{-\frac{1}{n}} = \prod_{i=1}^{n} \left(\frac{1}{P(w_i | w_1, \ldots, w_{i-1})} \right)^{\frac{1}{n}}$$

- It is actually equal to the exponential of the cross-entropy loss:

$$PP = \prod_{i=1}^{n} \left(\frac{1}{\hat{y}_{w_{i+1}}} \right)^{\frac{1}{n}} = exp\left(\frac{1}{n} \sum_{i=1}^{n} -log(\hat{y}_{w_{i+1}}) \right) = exp(\mathcal{L})$$

- So *lower perplexity == lower loss == higher data likelihood*.